

Ona Analytics

SEE THE SURGE. SECURE THE SUPPLY.

Predictive Demand & Supply Chain Radar for Remote Eco-Lodges

Architected & Developed Solo By: Eugene Mutembei
Frontend Developer & AI Systems Engineer
Hack the Zero Stack (Vercel + AWS) | June 2026

1. Executive Summary

Ona (*verb, Swahili*): To see, perceive, or understand.

Ona Analytics is a serverless, AI-native demand radar built specifically for remote safari camps and eco-lodges (10-25 tented suites). Organizations operating in isolated locations like the Maasai Mara face a critical pain point: supply chain isolation. If a sudden 30% spike in weekend bookings occurs, supplies must be trucked in from 5 hours away. Under-forecasting means running out of fresh food and linens.

By leveraging the precise technical synergy of **Vercel's Edge Network** and **Amazon Aurora PostgreSQL**, Ona Analytics provides real-time predictive analytics and natural language business intelligence, enabling managers to optimize weekly supply trucks before it's too late.

2. Design System & Brand Identity

The visual identity revolves around the "Ona" concept. A minimalist, blue and black eye illustration serves as the primary logo, symbolizing vision and operational clarity. The UI is built using Next.js, Tailwind CSS, and shadcn/ui components.

Theme Architecture

The platform defaults to a clean light mode but features a meticulously designed dark mode to reduce eye strain during late-night operations management.

- **Dark Mode Base:** Deep Obsidian (bg-slate-950).
- **Primary Accent:** Electric Indigo (text-indigo-600).
- **Alerts/Spikes:** Rose (text-rose-600).
- **Logo Behavior:** The minimalist black/blue eye logo dynamically applies a white-scale filter (`dark:invert`) against the obsidian background.

Cross-Platform UX

Engineered for operations managers in the field, the dashboard utilizes a responsive mobile-first architecture.

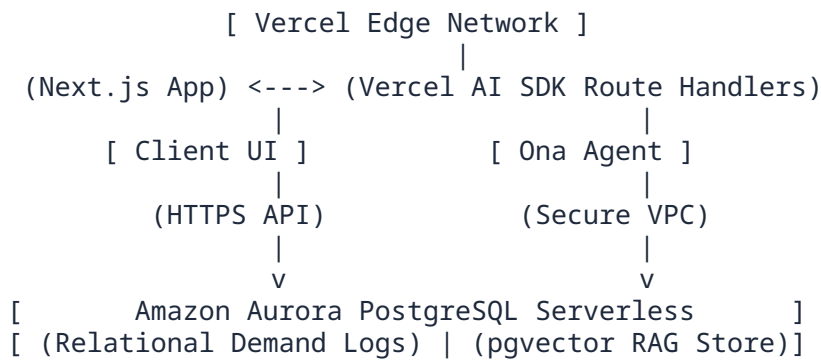
- **Desktop:** Persistent left-side navigation and a dense command center layout.
- **Mobile:** Navigation collapses into a native-app style bottom tab bar or a side-sheet drawer.
- **AI Agent:** Accessible via a floating action button (FAB) that slides out a chat interface.

3. Architecture & Database Strategy

Ona Analytics is built to fulfill the exact criteria of the *Hack the Zero Stack* evaluation. The choice of database was highly intentional.

Why Amazon Aurora PostgreSQL?

Justification: The split-brain AI architecture required seamless, zero-latency querying of strict relational occupancy metrics alongside unstructured, high-dimensional vector embeddings (`pgvector`) for the camp's logistical SOPs, all within a single serverless environment.



4. The Split-Brain AI Pipeline (Ona Agent)

The system combines deterministic SQL data retrieval with semantic Retrieval-Augmented Generation (RAG).

Deterministic Tool Calling

When predicting a surge (e.g., from a steady 41% to a 95% weekend spike), the AI uses Next.js server actions to execute raw SQL against the demand_logs table. This prevents hallucination of financial or capacity numbers.

Vector Context (pgvector)

When asked logistical questions, the AI embeds the query and searches the context_knowledge table using HNSW cosine distance calculations. This pulls localized Standard Operating Procedures (SOPs).

Example Interaction Flow

User: "Ona, we have a booking surge this weekend hitting 95%. What do I need to do about supplies?"

Ona Agent: "Based on SOPs and weather forecasts, you need +40kg of fresh produce and 10 gas cylinders. Due to forecasted rain on the Sekenani road, contact Narok suppliers to dispatch the truck 12 hours early on Wednesday."

5. Database Schema & Demo Seed Data

```
-- 1. Enable Native Vector Search
CREATE EXTENSION IF NOT EXISTS pgvector;

-- 2. Relational Data (The Occupancy Spike)
CREATE TABLE demand_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    org_id UUID NOT NULL,
    log_date DATE NOT NULL,
    metric_type VARCHAR(50) NOT NULL,
    actual_value DECIMAL(12, 4) NOT NULL,
    predicted_value DECIMAL(12, 4)
);

INSERT INTO demand_logs (org_id, log_date, metric_type, actual_value, predicted_value)
VALUES
('11111111-1111-1111-1111-111111111111', CURRENT_DATE - INTERVAL '7 days', 'occupancy_rate',
41.0, 40.0),
('11111111-1111-1111-1111-111111111111', CURRENT_DATE + INTERVAL '3 days', 'occupancy_rate',
95.5, 96.0); -- THE SPIKE

-- 3. High-Dimensional Vector Embeddings (The Logistics SOPs)
CREATE TABLE context_knowledge (
    id UUID PRIMARY KEY,
    org_id UUID NOT NULL,
    content TEXT NOT NULL,
    embedding VECTOR(1536) NOT NULL
);

CREATE INDEX idx_vector_hnsw
ON context_knowledge USING hnsw (embedding vector_cosine_ops);

-- Insert localized knowledge (e.g., Sekenani road weather warnings)
INSERT INTO context_knowledge (id, org_id, content, embedding)
VALUES (uuid_generate_v4(), '11111111-1111-1111-1111-111111111111',
'Weather SOP: If heavy rain is forecasted, supply trucks must depart 12 hours earlier due to
severe mud on the Sekenani gate road.',
'[0.1, 0.2, ...]':vector);
```